Réseau de Hopfield et Classification

L'étude précédente a montré que le réseau de Hopfield constitue un modèle de mémoire associative. Le développement de ce projet a pour but d'exploiter cette propriété pour résoudre un problème concret, fréquent en intelligence artificielle : la *classification*. Ici, le problème de classification s'appliquera à la *reconnaissance de chiffres manuscrits dans des images*.

1 Classification, Reconnaissance d'images

Le problème de la classification est le suivant : on dispose d'un ensemble d'éléments (ici, des images), pouvant appartenir à différentes catégories (ici, les chiffres entre 0 et 9), et l'on cherche à attribuer la catégorie appropriée à chaque élément (ici, affecter la catégorie '0' à toutes les images représentant un zéro, etc).

Pour cela, on utilise un algorithme de classification (ici, le réseau de Hopfield). Celui-ci est chargé de prendre en entrée chaque élément et de renvoyer la catégorie appropriée, au terme d'un calcul (ici, l'évolution du réseau vers un souvenir). Le calcul ne se fonde que sur certaines propriétés simples des éléments d'entrée (ici, les valeurs des pixels de l'image).

Pour parvenir à réaliser cette tâche, la première phase est l'*apprentissage*. Elle consiste à configurer les paramètres de l'algorithme (ici, la matrice des poids) de sorte à optimiser la classification. En d'autres termes, cette étape vise à trouver les paramètres tels que l'algorithme détermine la vraie catégorie le plus souvent possible. Pour cela, on travaille sur des *données d'entrainement* : un échantillon des éléments, dont les catégories sont connues. Connaissant les vraies catégories, il est donc possible de trouver les paramètres de l'algorithme qui minimisent les erreurs de classification sur ce sous-ensemble des éléments.

La seconde phase est la classification en elle-même, qui est l'occasion de l'*évaluation* des performances du réseau. L'algorithme optimisé précédemment calcule les catégories des *données de test*, c'est-à-dire celles qui n'ont pas été utilisés dans la phase d'apprentissage. Les prédictions de l'algorithme sont confrontées aux vraies catégories, ce qui permet de vérifier sa fiabilité.

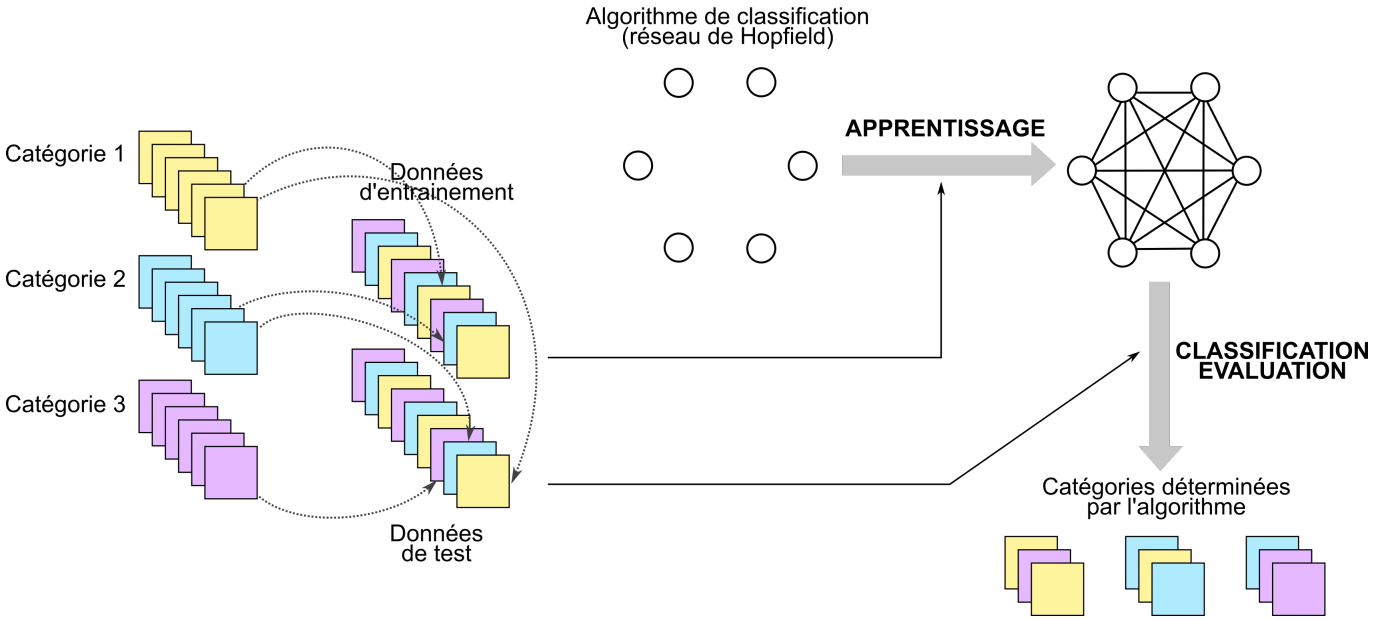


FIGURE 1 – Principe de la classification.

2 Données à classifier

2.1 Jeu de données MNIST

La base de données MNIST (Mixed National Institute of Standards and Technology), est regroupe un ensemble d'images représentant des chiffres écrits à la main (10 catégories, entre 0 et 9). Ce jeu de données est très utilisé en intelligence artificielle pour évaluer les performances de différents algorithmes sur des tâches de classification.



FIGURE 2 – Echantillon de la base MNIST.

Chaque image de la base de donnée est constituée de 64 pixels (8×8), en niveaux de gris.

Chaque pixel contient une valeur entre 0 et 16 indiquant l'intensité du niveau de gris.

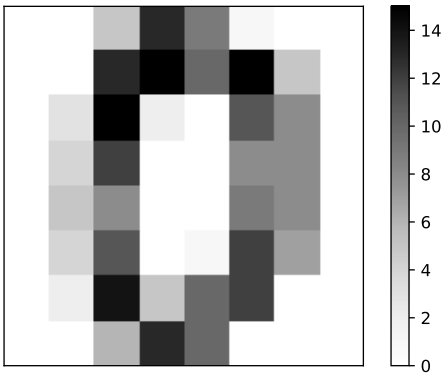


FIGURE 3 – Exemple d'image de la catégorie '0'.

2.2 Importation des données

Dans ce projet, vous disposez de deux fichiers contennant les données :

Images	`data.cvs`	Matrice de dimensions (1797, 64)	Chaque ligne correspond à une image. Chaque colonne correspond à un pixel.
Catégories	`labels.cvs`	Matrice de dimensions (1797,)	Chaque élément d'indice `k` indique la catégorie de l'image `data[k,:]` (k^{eme} ligne).

1. Importer les données à l'aide de la syntaxe suivante :

```
data = np.genfromtxt('data.csv', delimiter=',')
labels = np.genfromtxt('data.csv', delimiter=',')
```

2. Manipuler les données, pour s'assurer de bien comprendre leur format : examiner les dimensions des matrices et le contenu des premières lignes...

2.3 Séparation des données

Par la suite, il sera utile de regrouper les données appartiennant à une même catégorie.

Créer une fonction extrayant de la matrice `data` une sous-matrice contenant uniquement les lignes de la catégorie n (avec n le chiffre d'intérêt).

D'autre part, les données doivent être réparties en deux sous-ensembles : les *données d'entrainement*, réservées à l'apprentissage, et les *données de test*, réservées à l'évaluation des performances de l'algorithme après apprentissage.

Créer une fonction constituant les ensemble d'apprentissage et de test, sous la forme de sous-matrices de la matrice totale. Cette fonction pourra admettre un argument déterminant la proportion de données dans l'ensemble d'apprentissage.

3 Apprentissage

3.1 Binarisation des images

Pour l'instant, les données brutes de la base MNIST ne peuvent pas encore être classifiées par un réseau de Hopfield. En effet, les images présentées au réseau doivent être dans un format binaire, ne contenant que des valeurs -1 ou 1. Or, les images brutes sont encore en niveaux de gris.

Pour les convertir en images binaires, on peut réaliser un seuillage. On pourra déterminer visuellement une valeur du seuil pertinent.

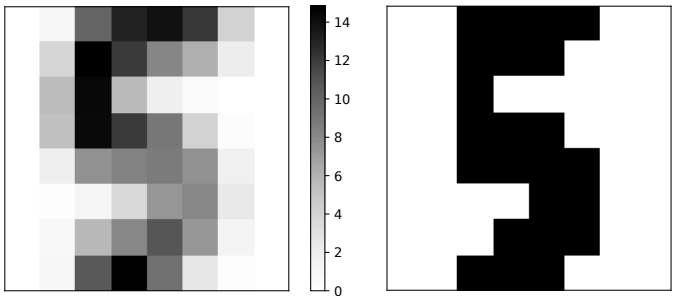


FIGURE 4 – Exemple de binarisation.

Créer une fonction binarisant une image (valeurs -1 ou 1) en appliquant un seuil.
Tester différents seuils en observant l'image résultante.

3.2 Construction des souvenirs

Le réseau de Hopfield offre une manière simple de déterminer la catégorie d'une image. Pour cela, on peut faire mémoriser au réseau un souvenir de référence représentant chaque catégorie. De cette manière, lorsque le réseau sera confronté à une image quelconque, il convergera vers l'un des souvenirs, indiquant la classe de celle-ci.

Dans le cas du réseau de Hopfield, l'apprentissage peut donc se dérouler en deux étapes :
1. Elaborer des 'souvenirs' de référence, un pour chaque catégorie. Pour cela, on pourra simplement définir le souvenir associé à une catégorie comme l'image moyenne de toutes les images de cette catégorie (parmi les données d'entrainement).
2. Etablir la matrice des poids W correspondant à cet ensemble de souvenirs.

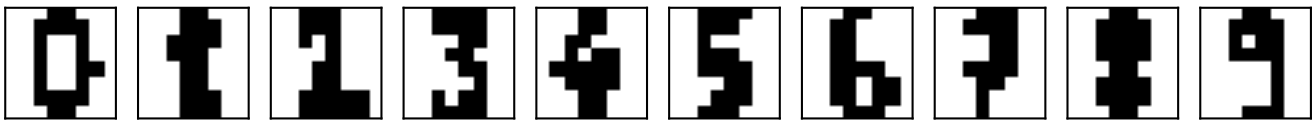


FIGURE 5 – Exemples de 'souvenirs' pour chaque catégorie (image moyenne binarisée).

Dans un premier temps, on va simplifier le problème en se restreignant à deux catégories, par exemple '0' et '1'. En effet, choisir deux chiffres au lieu de 10 permet de limiter l'émergence de souvenirs parasites et d'améliorer les performances de la classification.

Créer une fonction qui construise le souvenir de référence d'une catégorie. Cette fonction pourra procéder de la manière suivante :
1. Sélectionner parmi les *données d'entrainement* les images correspondant à la cétégorie souhaitée.
2. *Avant d'appliquer le seul*, moyenner toutes les images répertoriées.
3. Appliquer le seuil à l'image moyenne obtenue.

Créer ensuite la matrice des poids à l'aide d'une fonction du sujet précédent.

4 Evaluation des performances

Désormais, on va étudier la capacité du réseau de Hopfield à classifier les images de l'*ensemble de test*.

4.1 Classification d'une image

Pour classifier une image inconnue, on place le réseau dans un état initial correspondant à cette image, puis on le laisse évoluer jusqu'à ce qu'il atteigne un état stable. La détermination de la catégorie de l'image inconnue est alors immédiate, en examinant l'état final :

— Dans l'idéal, l'état final atteint est exactement un souvenir de référence associé à une catégorie. Le réseau attribue alors à l'image inconnue la catégorie correspondante.
— Néanmoins, il pourra arriver que le réseau évolue vers le souvenir parasite. Dans ce cas, il échoue à affecter l'image à une catégorie.

Le réseau aura commis une erreur de classification dès lors que la catégorie identifiée diffère de la vraie catégorie de l'image, ou bien lorsqu'aucune catégorie n'est attribuée.

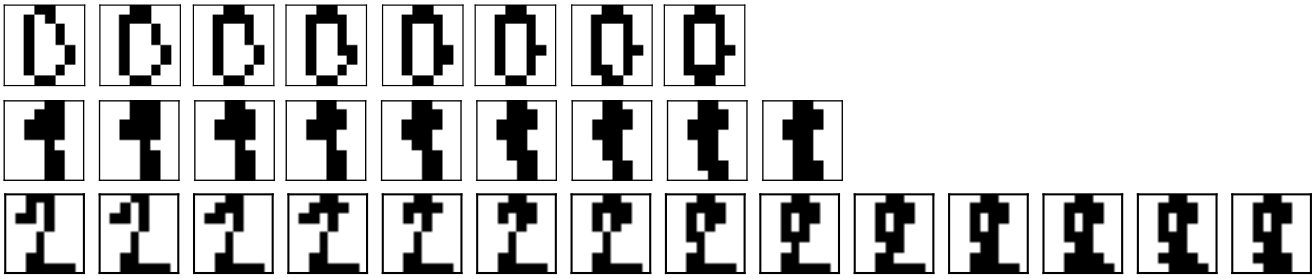


FIGURE 6 – Processus de classification par le réseau de Hopfield.
Les deux premières lignes montrent le résultat d'une classification binaire, ne faisant intervenir que les catégories '0' et '1'. Pour deux images de l'ensemble de test (à gauche), le réseau atteint le souvenir de référence correspondant à la catégorie appropriée. La troisième ligne illustre un cas où le réseau converge vers un souvenir parasite au lieu de la catégorie '2'. Cet exemple est issu d'une classification faisant intervenir les catégories '0', '1' et '2'. De manière générale, accroître le nombre de catégories fait émerger davantage de souvenirs parasites.

Créer une fonction qui identifie la catégorie d'une image. Pour cela, cette fonction devra comparer l'état final du réseau aux différents souvenirs de référence.

4.2 Matrice de confusion

En intelligence artificielle, une méthode systématique pour évaluer les performances d'un algorithme de classification consiste à construire une *matrice de confusion*.

Dans cette matrice, chaque ligne correspond à une vraie catégorie, chaque colonne correspond à une catégorie prédite par l'algorithme. La cellule située sur la ligne i et la colonne j contient le nombre d'éléments appartenant à la vraie catégorie i qui ont été affectés à la catégorie j par l'algorithme.

Un algorithme performant est caractérisé par une matrice de confusion dans laquelle les coefficients diagonaux sont maximaux, et les autres coefficients minimaux (nuls dans le cas d'un algorithme parfait). En effet, cela indique que pour toutes les catégories, les images appartenant à celle-ci ont effectivement été affectés à cette même catégorie par l'algorithme.

Cette matrice offre aussi la possibiilité d'identifier les catégories suscitant le plus de difficultés de classification. Par exemple, certains chiffres présentant plus de similarité pourraient s'avérer plus difficiles à distinguer par l'algorithme.

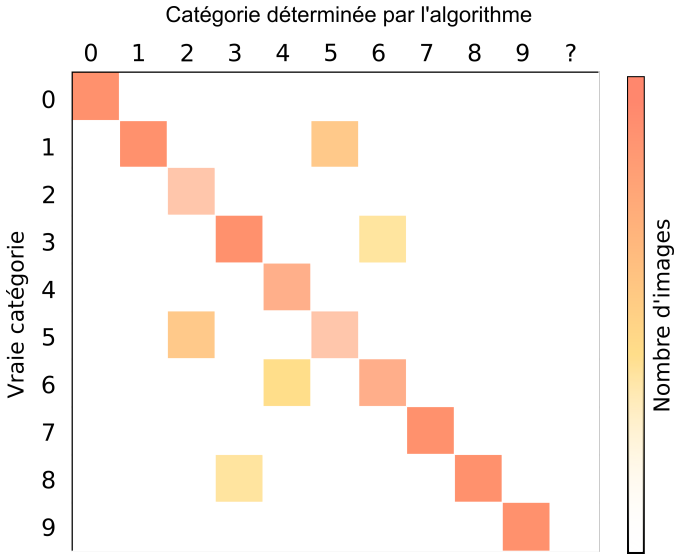


FIGURE 7 – Exemples de matrice de confusion.
Cette matrice de dimensions $(10, 11)$ représente les performances d'une classification réalisée pour 10 chiffres. La dernière colonne '?' correspond aux cas où le réseau échoue à classifier une image, car il atteint un souvenir parasite. Pour une classification faisant intervenir seulement k classes, les dimensions de la matrice seraient réduites à $(k, k + 1)$.

1. Créer une fonction qui construise la matrice de confusion en prenant en compte l'ensemble des images de l'ensemble de test.

2. Evaluer les performances de l'algorithme en fonction du nombre de classes retenues.

3. Une classification binaire est-elle aussi simple quelle que soit la paire de chiffres choisis ? Interpréter en termes de similarités entre chiffres.